Agent Swarm - Multi-Tenant Architecture

Fecha: 2026-03-21 **Autor:** Cesar / Claude Code **Estado:** Diseno de arquitectura **Principio:** Un swarm por workspace. Multiples empresas, equipos y proyectos aislados.

---

1. Vision Multi-Tenant

El Agent Swarm no es solo para BAAT Digital. Es una plataforma que cualquier empresa puede usar para coordinar desarrollo con agentes IA.

```
Empresa A (baatdigital)
  ├── Workspace "SuperPago"     → 3 nodos trabajando
  ├── Workspace "Marketing"     → 1 nodo trabajando
  └── Workspace "AlertaTribunal"→ 0 nodos (idle)

Empresa B (otra-empresa)
  ├── Workspace "SaaS Principal"→ 5 nodos trabajando
  └── Workspace "Mobile App"    → 2 nodos trabajando

Empresa C (freelancer)
  └── Workspace "Mi Proyecto"   → 1 nodo trabajando
```

Aislamiento total: Empresa A nunca ve tareas, nodos, contratos ni learnings de Empresa B.

---

2. Jerarquia

```
┌──────────────────────────────────────────────────────────────┐
│                        TENANT                                 │
│   (empresa/organizacion GitHub)                               │
│   Ejemplo: baatdigital                                        │
│                                                               │
│   ┌──────────────────────────────────────────────────────┐   │
│   │                  WORKSPACE                           │   │
│   │   (producto/proyecto con su GitHub Project)          │   │
│   │   Ejemplo: SuperPago                                 │   │
│   │                                                      │   │
│   │   ┌──────────────────────────────────────────┐       │   │
│   │   │                SWARM                     │       │   │
│   │   │   (grupo de nodos asignados)             │       │   │
│   │   │                                          │       │   │
│   │   │   ┌───────┐  ┌───────┐  ┌───────┐       │       │   │
│   │   │   │Node 1│  │Node 2│  │Node 3│       │       │   │
│   │   │   └───────┘  └───────┘  └───────┘       │       │   │
│   │   └──────────────────────────────────────────┘       │   │
│   │                                                      │   │
│   │   Tasks, Contracts, Learnings, Messages              │   │
│   └──────────────────────────────────────────────────────┘   │
│                                                               │
```

```
│   ┌──────────────────────────────────────────────────┐    │
│   │                 WORKSPACE                        │    │
│   │  Ejemplo: Marketing                              │    │
│   │   ┌───────┐                                      │    │
│   │   │Node 4│  (1 nodo, trabaja solo)               │    │
│   │   └───────┘                                      │    │
│   └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘
```

Relaciones

Concepto	Cardinalidad	Ejemplo
Tenant → Workspaces	1:N	baatdigital → SuperPago, Marketing, AlertaTribunal
Workspace → GitHub Project	1:1	SuperPago → projects/1
Workspace → Repos	1:N	SuperPago → covacha-payment, mf-core, ...
Workspace → Swarm	1:1	SuperPago → {nodo-1, nodo-2, nodo-3}
Node → Workspaces	N:M	Un nodo puede estar en multiples workspaces
Node → Tenant	N:1	Un nodo pertenece a 1 tenant (pero multiples workspaces)

---

3. DynamoDB Schema Multi-Tenant

3.1 Partition Key con scope de tenant + workspace

Patron: Prefijo `{TENANT}#{WORKSPACE}#` en todo.

```
covacha-agent-memory (single-table, PAY_PER_REQUEST)
│
│ ─── TENANT CONFIG ───
│
├── TENANT#baatdigital│CONFIG
│   → org_name, github_org, plan, max_nodes, max_workspaces,
│     created_at, admin_emails, billing_status
│
├── TENANT#baatdigital│WORKSPACE#superpago
│   → workspace_name, github_project_id, github_project_number,
│     status_field_id, status_options{todo,in_progress,done},
│     producto_field_id, repos[], active, created_at
│
├── TENANT#baatdigital│WORKSPACE#marketing
│   → (misma estructura)
│
│ ─── NODOS (scoped por tenant) ───
│
├── TENANT#baatdigital│NODE#cesar-macbook-pro
│   → node_id, capabilities, repos, roles, status,
│     current_workspace, current_task, last_heartbeat,
│     workspaces_joined[], ttl
```

```
│
│ ─── TAREAS (scoped por tenant+workspace) ───
│
├── TENANT#baatdigital│WS#superpago│TASK#043│META
│   → number, title, labels, status, repo, branch,
│     github_node_id, recommended_model, assigned_node
│
├── TENANT#baatdigital│WS#superpago│TASK#043│LOCK
│   → locked_by (node_id), locked_at, ttl
│
│ ─── DEPENDENCIAS (scoped por workspace) ───
│
├── TENANT#baatdigital│WS#superpago│DEP#044│DEPENDS_ON#043
│   → blocker_task, blocker_status, blocked_task
│
│ ─── MENSAJES (scoped por workspace) ───
│
├── TENANT#baatdigital│WS#superpago│MSG#1711036800-cesar-macbook-pro
│   → from_node, type, task_id, payload, read_by[], ttl
│
│ ─── CONTRATOS (scoped por workspace) ───
│
├── TENANT#baatdigital│WS#superpago│CONTRACT#spei-transfer-v1│META
│   → method, path, request_schema, response_schema,
│     published_by, task_id
│
│ ─── LEARNINGS (scoped por workspace+modulo) ───
│
├── TENANT#baatdigital│WS#superpago│LEARNING#covacha-payment│patterns
│   → gotchas[], updated_by, updated_at
│
│ ─── SYNC (scoped por workspace) ───
│
└── TENANT#baatdigital│WS#superpago│SYNC#github│META
    → last_sync, errors[]
```

3.2 GSIs para queries eficientes

```
GSI-1: tenant-workspace-status
  PK: TENANT#{tenant}│WS#{workspace}
  SK: status
  → "Dame todas las tareas todo de SuperPago"

GSI-2: node-lookup
  PK: TENANT#{tenant}│NODE#{node_id}
  SK: (primary SK)
  → "En que workspace esta trabajando este nodo?"

GSI-3: sync-tracker
  PK: SYNC#github
  SK: TENANT#{tenant}│WS#{workspace}
  → "Cuando fue el ultimo sync de cada workspace?"
```

---

4. Configuracion: `.covacha-node.yml` Multi-Tenant

4.1 Archivo local del nodo

```
# ~/.covacha-node.yml
node_id: "cesar-macbook-pro"       # Auto-generado o manual

# Tenant (empresa)
tenant:
  id: "baatdigital"
  github_org: "baatdigital"

# Workspaces donde participa este nodo
workspaces:
  - id: "superpago"
    repos:
      - path: /Users/casp/sandboxes/superpago/covacha-payment
        name: covacha-payment
      - path: /Users/casp/sandboxes/superpago/covacha-core
        name: covacha-core
      - path: /Users/casp/sandboxes/superpago/mf-core
        name: mf-core
    capabilities: [backend, frontend, testing]
    roles: [developer, tech_lead]

  - id: "marketing"
    repos:
      - path: /Users/casp/sandboxes/superpago/mf-marketing
        name: mf-marketing
    capabilities: [frontend]
    roles: [developer]

# Config global del nodo
preferred_model: "sonnet"
max_concurrent_tasks: 1
auto_switch_workspace: true     # Cambiar de workspace si no hay tareas
```

4.2 Un nodo puede trabajar en multiples workspaces

```
Escenario: Nodo "cesar-macbook" esta en SuperPago y Marketing

T=0:00  Agent Loop busca tarea en SuperPago (workspace primario)
        → Encuentra ISS-043, lo reclama, trabaja

T=0:45  Termina ISS-043 en SuperPago
        → Busca siguiente en SuperPago... no hay tareas
        → auto_switch_workspace=true
        → Busca en Marketing... encuentra ISS-012
        → Cambia de workspace, reclama ISS-012

T=1:15  Termina ISS-012 en Marketing
        → Busca en Marketing... no hay
```

```
        → Busca en SuperPago... ISS-044 se desbloqueo!
        → Vuelve a SuperPago, reclama ISS-044
```

---

5. CLI: covacha Multi-Tenant

5.1 Setup de tenant (primera vez)

```
# Inicializar con un tenant existente
covacha init --tenant baatdigital --org baatdigital

# Output:
# ┌──────────────────────────────────────────────┐
# │ Covacha Agent Swarm - Setup                  │
# │                                              │
# │ Tenant: baatdigital                          │
# │ GitHub Org: baatdigital                      │
# │                                              │
# │ Workspaces disponibles:                      │
# │   [1] SuperPago (Project #1, 47 tareas)      │
# │   [2] Marketing (Project #3, 12 tareas)      │
# │   [3] AlertaTribunal (Project #4, 8 tareas)  │
# │                                              │
# │ Selecciona workspaces (comma-separated): 1,2 │
# │                                              │
# │ Repos detectados en tu maquina:              │
# │   ✓ covacha-payment → SuperPago              │
# │   ✓ covacha-core → SuperPago                 │
# │   ✓ mf-marketing → Marketing                 │
# │   ✗ mf-core (no clonado, necesario para front) │
# │                                              │
# │ Capabilities inferidas:                      │
# │   SuperPago: [backend, testing]              │
# │   Marketing: [frontend]                      │
# │                                              │
# │ Confirmar? [Y/n]                             │
# └──────────────────────────────────────────────┘
```

5.2 Crear tenant nuevo (otra empresa)

```
# Una empresa nueva quiere usar el swarm
covacha tenant create \
  --name "TechStartup" \
  --github-org "techstartup-dev" \
  --admin-email "cto@techstartup.com"

# Output:
# ✓ Tenant "techstartup" creado
# ✓ GitHub org "techstartup-dev" verificada (23 repos)
#
# Siguiente paso: crear workspaces
# → covacha workspace create --tenant techstartup --name "main-app"
```

5.3 Crear workspace

```
covacha workspace create \
  --tenant baatdigital \
  --name "inventario" \
  --github-project 5 \
  --repos covacha-inventory,mf-inventory

# Output:
# ✓ Workspace "inventario" creado en tenant "baatdigital"
# ✓ Vinculado a GitHub Project #5
# ✓ Repos: covacha-inventory, mf-inventory
# ✓ Sincronizando tareas... 23 tareas importadas
```

5.4 Comandos completos

```
# ─── TENANT ───
covacha tenant create   --name X --github-org Y --admin-email Z
covacha tenant list
covacha tenant info     --tenant X
covacha tenant delete   --tenant X  # Solo si no hay workspaces activos

# ─── WORKSPACE ───
covacha workspace create  --tenant X --name Y --github-project N --repo
covacha workspace list    --tenant X
covacha workspace info    --tenant X --workspace Y
covacha workspace delete  --tenant X --workspace Y
covacha workspace sync    --tenant X --workspace Y  # Force sync con Gi

# ─── NODE ───
covacha init              --tenant X --org Y          # Setup inicial
covacha start             [--workspace Z]             # Iniciar loop
covacha stop                                          # Parar gracefull
covacha status                                        # Status del nodo
covacha join              --workspace Z               # Unirse a otro w
covacha leave             --workspace Z               # Salir de un wor
covacha discover                                      # Re-detectar rep

# ─── SWARM ───
covacha swarm status      --tenant X [--workspace Y]  # Status de todos
covacha swarm nodes       --tenant X [--workspace Y]  # Lista de nodos
covacha swarm dashboard   --tenant X                  # Dashboard HTML

# ─── TASK ───
covacha task next                                     # Siguiente tarea
covacha task claim        --task N                    # Reclamar tarea
covacha task release      --task N --status done      # Liberar tarea
covacha task eval         --task N --type pre_release # Evaluar ticket

# ─── MSG ───
covacha msg send          --type X --task N --payload '{}'
covacha msg read          [--mark-read]
covacha msg broadcast     --text "mensaje libre"
```

6. GitHub Integration Multi-Tenant

6.1 Cada workspace tiene su propio GitHub Project

```
# Workspace SuperPago
github:
  org: "baatdigital"
  project_number: 1
  project_id: "PVT_kwDOBBTMcM4BPNTK"
  status_field_id: "PVTSSF_lADOBBTMcM4BPNTKzg9rxDE"
  status_options:
    todo: "f75ad846"
    in_progress: "47fc9ee4"
    done: "98236657"
  repos:
    - baatdigital/covacha-payment
    - baatdigital/covacha-core
    - baatdigital/mf-core

# Workspace Marketing (mismo org, diferente project)
github:
  org: "baatdigital"
  project_number: 3
  project_id: "PVT_kwDOBBTMcM4BXYZAB"
  status_field_id: "PVTSSF_..."
  repos:
    - baatdigital/mf-marketing
    - baatdigital/covacha-core

# Workspace de OTRA empresa
github:
  org: "techstartup-dev"
  project_number: 1
  project_id: "PVT_kwDOCCCCCC..."
  repos:
    - techstartup-dev/backend-api
    - techstartup-dev/frontend-web
```

6.2 Sync multi-workspace

```
def sync_all_workspaces(tenant_id: str) -> dict:
    """Sincroniza todos los workspaces de un tenant con GitHub."""
    workspaces = get_workspaces(tenant_id)
    results = {}

    for ws in workspaces:
        if not ws.get("active"):
            continue

        github_config = ws["github"]
        tasks = fetch_github_project_items(
```

```
            org=github_config["org"],
            project_id=github_config["project_id"],
            status_field_id=github_config["status_field_id"],
        )

        synced = 0
        errors = []
        for task in tasks:
            try:
                upsert_task(
                    tenant_id=tenant_id,
                    workspace_id=ws["id"],
                    task=task
                )
                synced += 1
            except Exception as e:
                errors.append(str(e))

        results[ws["id"]] = {"synced": synced, "errors": errors}

    return results
```

6.3 GitHub Token por tenant

```
# ~/.covacha-node.yml
tenant:
  id: "baatdigital"
  github_org: "baatdigital"
  github_token_env: "GH_TOKEN_BAAT"  # Variable de entorno con el token

# O usar gh CLI con multiple cuentas:
# gh auth switch --user cesar-baat
# gh auth switch --user cesar-techstartup
```

---

7. Workspace Discovery Automatico

7.1 Auto-detect workspaces desde GitHub

```
def discover_workspaces(github_org: str) -> list[dict]:
    """Descubre workspaces disponibles leyendo GitHub Projects de la or
    projects = list_org_projects(github_org)

    workspaces = []
    for project in projects:
        # Leer campos del project para inferir config
        fields = get_project_fields(project["id"])
        status_field = next((f for f in fields if f["name"] == "Status"

        ws = {
            "id": slugify(project["title"]),  # "SuperPago" → "superpag
            "name": project["title"],
```

```
            "github": {
                "org": github_org,
                "project_number": project["number"],
                "project_id": project["id"],
                "status_field_id": status_field["id"] if status_field e
                "status_options": extract_status_options(status_field),
            },
            "repos": extract_repos_from_project(project),
            "task_count": project["item_count"],
        }
        workspaces.append(ws)

    return workspaces
```

7.2 Auto-detect repos del workspace

```
def extract_repos_from_project(project: dict) -> list[str]:
    """Extrae repos unicos de los issues de un GitHub Project."""
    repos = set()
    items = get_project_items(project["id"])
    for item in items:
        if item.get("content", {}).get("repository"):
            repos.add(item["content"]["repository"]["nameWithOwner"])
    return sorted(repos)
```

---

8. Aislamiento de Datos

8.1 Reglas de aislamiento

```
1. TENANT BOUNDARY (maximo aislamiento)
   - Un nodo de tenant A NUNCA ve datos de tenant B
   - Todos los queries incluyen tenant_id en el PK
   - No existe query cross-tenant

2. WORKSPACE BOUNDARY (aislamiento dentro del tenant)
   - Tareas, contratos, deps son por workspace
   - Learnings son por workspace (pueden compartirse opcional)
   - Mensajes son por workspace
   - Nodos pueden estar en multiples workspaces del mismo tenant

3. NODE BOUNDARY (aislamiento por nodo)
   - Locks son exclusivos por nodo
   - Current_task es por nodo
   - Heartbeat es por nodo
```

8.2 Query patterns seguros

```
def get_available_tasks(tenant_id: str, workspace_id: str, **kwargs) ->
    """SIEMPRE requiere tenant_id + workspace_id. No hay queries global
    prefix = f"TENANT#{tenant_id}|WS#{workspace_id}|TASK#"
    # Scan con FilterExpression que incluye el prefix
```

```
    ...

def claim_task(tenant_id: str, workspace_id: str, task_id: str, node_id
    """Lock scoped por tenant+workspace."""
    table.put_item(
        Item={
            "PK": f"TENANT#{tenant_id}│WS#{workspace_id}│TASK#{task_id}
            "SK": "LOCK",
            "locked_by": node_id,
            ...
        },
        ConditionExpression=Attr("PK").not_exists(),
    )
```

8.3 Validacion en MCP Server

```
class CovachaMCPServer {
  private tenantId: string;
  private nodeId: string;

  constructor() {
    const config = loadNodeConfig();  // ~/.covacha-node.yml
    this.tenantId = config.tenant.id;
    this.nodeId = config.node_id;
  }

  // TODAS las operaciones inyectan tenant_id automaticamente
  async claimTask(taskId: string): Promise<Result> {
    const workspace = this.getCurrentWorkspace();
    return await dynamo.claimTask(
      this.tenantId,     // ← Siempre del config, nunca del input
      workspace.id,
      taskId,
      this.nodeId
    );
  }
}
```

---

9. Onboarding de Nueva Empresa

9.1 Flujo completo

```
# ─── PASO 1: Admin de la empresa crea el tenant ───
covacha tenant create \
  --name "MiEmpresa" \
  --github-org "miempresa-dev" \
  --admin-email "cto@miempresa.com" \
  --plan "team"

# Output:
# ✓ Tenant "miempresa" creado
```

```
# ✓ Plan: team (hasta 10 nodos, 5 workspaces)
# ✓ Token de invitacion: CVH-INVITE-a8f3b2c1

# ─── PASO 2: Descubrir workspaces desde GitHub ───
covacha workspace discover --tenant miempresa

# Output:
# GitHub Projects encontrados en miempresa-dev:
#   [1] "Backend API" (34 issues, 5 repos)
#   [2] "Mobile App" (21 issues, 3 repos)
#   [3] "Infrastructure" (8 issues, 2 repos)
#
# Crear workspaces? [1,2,3 / all / none]: all

# ✓ Workspace "backend-api" creado (34 tareas sincronizadas)
# ✓ Workspace "mobile-app" creado (21 tareas sincronizadas)
# ✓ Workspace "infrastructure" creado (8 tareas sincronizadas)

# ─── PASO 3: Cada dev de la empresa configura su maquina ───
# (en la maquina del dev)
covacha init --tenant miempresa --invite CVH-INVITE-a8f3b2c1

# Output:
# ✓ Conectado a tenant "miempresa"
# ✓ Node ID: juan-macbook-air
#
# Workspaces disponibles:
#   [1] backend-api (3 nodos activos)
#   [2] mobile-app (1 nodo activo)
#   [3] infrastructure (0 nodos)
#
# Selecciona workspaces: 1,2
#
# Repos detectados:
#   ✓ miempresa-api → backend-api
#   ✓ miempresa-mobile → mobile-app
#
# ✓ Nodo registrado. Ejecuta: covacha start

# ─── PASO 4: Iniciar loop ───
covacha start

# ✓ Conectado al swarm de "miempresa"
# ✓ 4 nodos activos en backend-api
# ✓ 2 nodos activos en mobile-app
# ✓ Buscando tarea...
```

9.2 Planes (escalabilidad)

Plan	Max Nodos	Max Workspaces	Sync Interval	Precio
free	2	1	15 min	$0
team	10	5	5 min	-

business	50	20	2 min	-
enterprise	Ilimitado	Ilimitado	1 min	-

Nota: Los precios son placeholder. El unico costo real es DynamoDB (~$1-5/mes).

---

10. Diagrama Completo

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              COVACHA PLATFORM                            │
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │  TENANT: baatdigital (GitHub: baatdigital)                       │   │
│   │                                                                  │   │
│   │   ┌──────────────────────────┐   ┌──────────────────────────┐   │   │
│   │   │ WS: superpago            │   │ WS: marketing            │   │   │
│   │   │ Project #1 (47 tasks)    │   │ Project #3 (12)          │   │   │
│   │   │                          │   │                          │   │   │
│   │   │ ┌────┐ ┌────┐ ┌────┐    │   │ ┌────┐                   │   │   │
│   │   │ │N1  │ │N2  │ │N3  │    │   │ │N1  │ (shared)          │   │   │
│   │   │ │back│ │fron│ │test│    │   │ │fron│                   │   │   │
│   │   │ └────┘ └────┘ └────┘    │   │ └────┘                   │   │   │
│   │   │                          │   │                          │   │   │
│   │   │ Tasks|Deps|Msgs|Contracts│   │ Tasks|Msgs|Learn         │   │   │
│   │   └──────────────────────────┘   └──────────────────────────┘   │   │
│   └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │  TENANT: techstartup (GitHub: techstartup-dev)                   │   │
│   │                                                                  │   │
│   │   ┌──────────────────────────┐   ┌──────────────────────────┐   │   │
│   │   │ WS: main-app             │   │ WS: mobile               │   │   │
│   │   │ Project #1 (34 tasks)    │   │ Project #2 (21)          │   │   │
│   │   │                          │   │                          │   │   │
│   │   │ ┌────┐ ┌────┐ ┌────┐    │   │ ┌────┐ ┌────┐            │   │   │
│   │   │ │N4  │ │N5  │ │N6  │    │   │ │N7  │ │N8  │            │   │   │
│   │   │ └────┘ └────┘ └────┘    │   │ └────┘ └────┘            │   │   │
│   │   └──────────────────────────┘   └──────────────────────────┘   │   │
│   └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
│   ┌──────────────────────────────────┐                                   │
│   │  DynamoDB: covacha-agent-memory  │                                   │
│   │  (single table, multi-tenant)    │                                   │
│   │  PK: TENANT#{id}|WS#{id}|...     │                                   │
│   └──────────────────────────────────┘                                   │
└──────────────────────────────────────────────────────────────────────────┘
```

---

11. Migracion desde Sistema Actual

11.1 Que cambia

Antes (single-tenant)	Despues (multi-tenant)
TASK#043|META	TENANT#baatdigital|WS#superpago|TASK#043|META
TEAM#backend-mac-1|STATUS	TENANT#baatdigital|NODE#cesar-macbook-pro
LEARNING#covacha-payment|patterns	TENANT#baatdigital|WS#superpago|LEARNING#covacha-payment|patterns
MSG#timestamp-mac-1|BROADCAST	TENANT#baatdigital|WS#superpago|MSG#timestamp-cesar-macbook|BROADCAST
SYNC#github|META	TENANT#baatdigital|WS#superpago|SYNC#github|META
config.py: GITHUB_ORG = "baatdigital"	~/.covacha-node.yml: tenant.github_org
config.py: GITHUB_PROJECT_ID = "PVT_..."	Workspace config en DynamoDB

11.2 Script de migracion

```
def migrate_to_multi_tenant(
    tenant_id: str = "baatdigital",
    workspace_id: str = "superpago"
) -> None:
    """Migra datos existentes al schema multi-tenant."""

    table = get_dynamo_client()

    # 1. Leer todos los items actuales
    all_items = full_table_scan(table)

    for item in all_items:
        old_pk = item["PK"]
        old_sk = item["SK"]

        # 2. Mapear al nuevo schema
        if old_pk.startswith("TASK#"):
            new_pk = f"TENANT#{tenant_id}|WS#{workspace_id}|{old_pk}"
        elif old_pk.startswith("TEAM#"):
            # Extraer machine name → convertir a NODE#
            node_id = derive_node_id(old_pk)
            new_pk = f"TENANT#{tenant_id}|NODE#{node_id}"
        elif old_pk.startswith("LEARNING#"):
            new_pk = f"TENANT#{tenant_id}|WS#{workspace_id}|{old_pk}"
        elif old_pk.startswith("MSG#"):
            new_pk = f"TENANT#{tenant_id}|WS#{workspace_id}|{old_pk}"
        elif old_pk.startswith("SYNC#"):
            new_pk = f"TENANT#{tenant_id}|WS#{workspace_id}|{old_pk}"
        else:
            continue
```

```
        # 3. Escribir con nuevo PK
        new_item = {**item, "PK": new_pk}
        table.put_item(Item=new_item)

        # 4. Borrar el viejo (opcional, o dejar que TTL lo limpie)

    # 5. Crear registros de tenant y workspace
    create_tenant_config(tenant_id, "baatdigital")
    create_workspace_config(tenant_id, workspace_id, {
        "github_project_id": "PVT_kwDOBBTMcM4BPNTK",
        "status_field_id": "PVTSSF_lADOBBTMcM4BPNTKzg9rxDE",
        # ... demas config de config.py actual
    })
```

11.3 Backward compatibility

```
# dynamo_client.py — wrapper que detecta schema viejo vs nuevo

def get_task(task_id: str, tenant_id: str = None, workspace_id: str = N
    """Compatible con ambos schemas durante migracion."""
    if tenant_id and workspace_id:
        pk = f"TENANT#{tenant_id}|WS#{workspace_id}|TASK#{task_id}"
    else:
        pk = f"TASK#{task_id}"  # Legacy fallback
    return table.get_item(Key={"PK": pk, "SK": "META"}).get("Item")
```

---

12. Covacha MCP Server Multi-Tenant

12.1 Config del MCP server

```
{
  "mcpServers": {
    "covacha": {
      "command": "covacha-mcp-server"
    }
  }
}
```

Zero env vars. El server lee `~/.covacha-node.yml` que ya tiene tenant, workspaces, y node_id.

12.2 Tools con scope automatico

```
// Cada tool recibe tenant+workspace del contexto del nodo
// El usuario NO necesita pasarlos como parametros

covacha_claim_task({ task_id: "043" })
// Internamente: tenant=baatdigital, workspace=superpago (del config)

covacha_swarm_status({})
// Internamente: muestra nodos del tenant actual
```

```
covacha_workspace_switch({ workspace: "marketing" })
// Cambia el workspace activo del nodo
```

12.3 Tools nuevas para multi-tenant

```
// ─── Workspace Management ───
covacha_workspace_list({})
// Lista workspaces del tenant actual

covacha_workspace_switch({ workspace: "marketing" })
// Cambia workspace activo

covacha_workspace_create({
  name: "nuevo-proyecto",
  github_project_number: 5,
  repos: ["miempresa/repo-a", "miempresa/repo-b"]
})
// Crea workspace nuevo

covacha_workspace_sync({ workspace: "superpago" })
// Fuerza sync con GitHub

// ─── Cross-Workspace ───
covacha_workspace_status({ all: true })
// Status de todos los workspaces del tenant

covacha_share_learning({
  from_workspace: "superpago",
  to_workspace: "marketing",
  learning: "Usar Decimal para montos"
})
// Compartir un learning entre workspaces
```

---

13. Escenarios Multi-Tenant

13.1 Agencia de desarrollo (1 empresa, multiples clientes)

```
Tenant: "mi-agencia"
  ├── WS: "cliente-a-ecommerce"  → 2 nodos
  ├── WS: "cliente-b-saas"      → 3 nodos
  └── WS: "interno-tools"       → 1 nodo

Los nodos se mueven entre workspaces segun demanda.
Learnings de un cliente NO se comparten con otros (aislamiento).
```

13.2 Startup con multiples productos (1 empresa, multiples products)

```
Tenant: "techstartup"
  ├── WS: "api-core"            → 2 nodos backend
  ├── WS: "mobile-app"          → 2 nodos frontend
```

```
  └── WS: "infra"               → 1 nodo ops
```

Nodos pueden compartir learnings entre workspaces del mismo tenant.

13.3 BAAT Digital (caso actual, migrado)

```
Tenant: "baatdigital"
  ├── WS: "superpago"           → 3 nodos (core product)
  ├── WS: "marketing"           → 1 nodo
  ├── WS: "alerta-tribunal"     → 1 nodo
  ├── WS: "dashboard"           → 0 nodos (sin trabajo pendiente)
  ├── WS: "ia-bots"             → 0 nodos
  └── WS: "inventario"          → 0 nodos
```

auto_switch_workspace=true:
Cuando SuperPago no tiene tareas, nodos migran a Marketing automaticame